EEPROM底层

官方参考

官方会给我们一个底层用于参考（其实就是iic总线的通信底层，这个底层我们不用去更改任何东西）

```c
/*  #   I2C代码片段说明
    1.  本文件夹中提供的驱动代码供参赛选手完成程序设计参考。
    2.  参赛选手可以自行编写相关代码或以该代码为基础，根据所选单片机类型、运行速度和试题
        中对单片机时钟频率的要求，进行代码调试和修改。
*/

#define DELAY_TIME  10 //如果出现问题，可以修改为5

//
static void I2C_Delay(unsigned char n)
{
    do
    {
        _nop_();_nop_();_nop_();_nop_();_nop_();
        _nop_();_nop_();_nop_();_nop_();_nop_();
        _nop_();_nop_();_nop_();_nop_();_nop_();
    }
    while(n--);
}

//
void I2CStart(void)
{
    sda = 1;
    scl = 1;
    I2C_Delay(DELAY_TIME);
    sda = 0;
    I2C_Delay(DELAY_TIME);
    scl = 0;
}

//
void I2CStop(void)
{
    sda = 0;
```

```
    scl = 1;
    I2C_Delay(DELAY_TIME);
    sda = 1;
    I2C_Delay(DELAY_TIME);
}

//
void I2CSendByte(unsigned char byt)
{
    unsigned char i;

    for(i=0; i<8; i++){
        scl = 0;
        I2C_Delay(DELAY_TIME);
        if(byt & 0x80){
            sda = 1;
        }
        else{
            sda = 0;
        }
        I2C_Delay(DELAY_TIME);
        scl = 1;
        byt <<= 1;
        I2C_Delay(DELAY_TIME);
    }

    scl = 0;
}

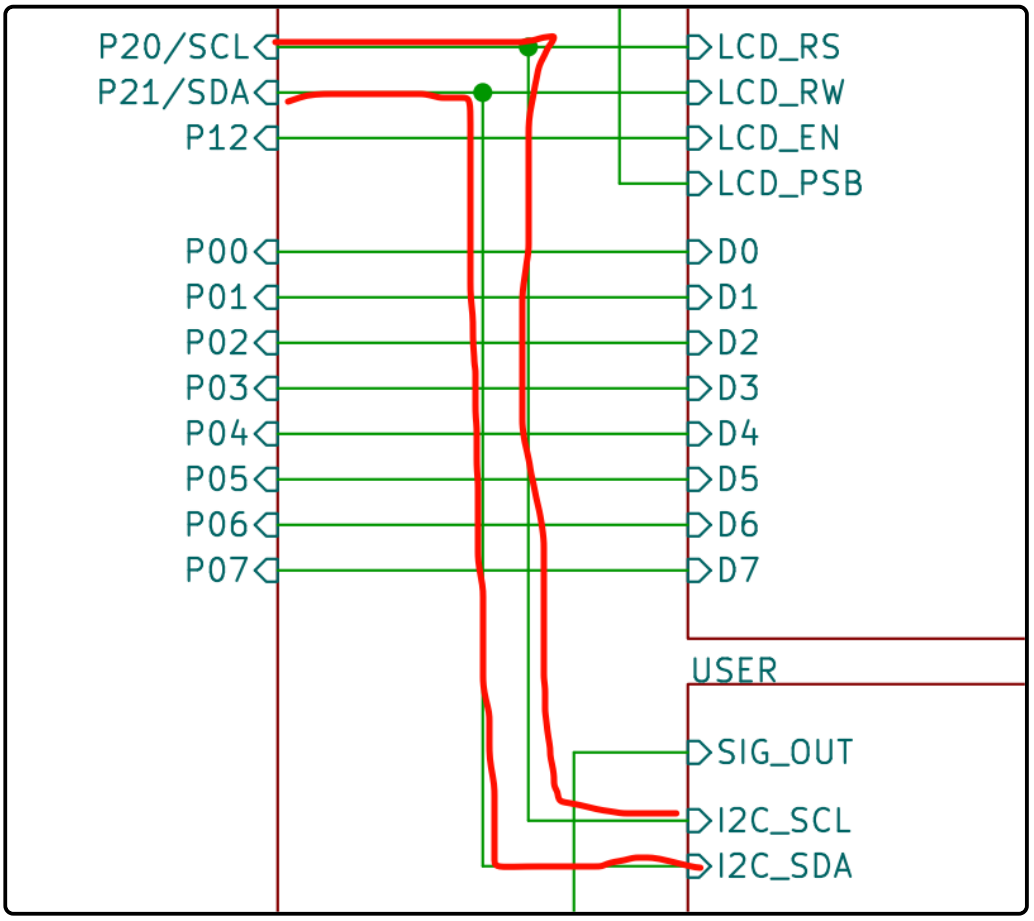
//
unsigned char I2CReceiveByte(void)
{
    unsigned char da;
    unsigned char i;
    for(i=0;i<8;i++){
        scl = 1;
        I2C_Delay(DELAY_TIME);
        da <<= 1;
        if(sda)
            da |= 0x01;
        scl = 0;
        I2C_Delay(DELAY_TIME);
    }
    return da;
}

//
unsigned char I2CWaitAck(void)
{
    unsigned char ackbit;

```

```
87      scl = 1;
88      I2C_Delay(DELAY_TIME);
89      ackbit = sda;
90      scl = 0;
91      I2C_Delay(DELAY_TIME);
92
93      return ackbit;
94  }
95
96  //
97  void I2CSendAck(unsigned char ackbit)
98  {
99      scl = 0;
100     sda = ackbit;
101     I2C_Delay(DELAY_TIME);
102     scl = 1;
103     I2C_Delay(DELAY_TIME);
104     scl = 0;
105     sda = 1;
106     I2C_Delay(DELAY_TIME);
107 }
108
109
```

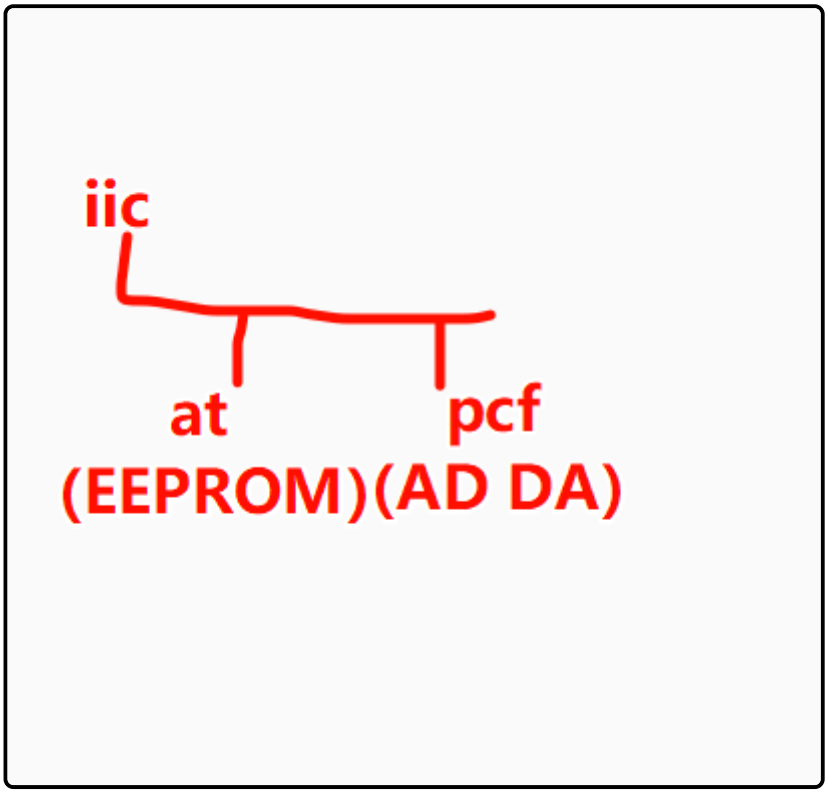
我们可以发现，底层中有scl，sda两个线没有定义，并且里面用到了nop函数，所以我们需要自己定义一下引脚和引入nop的头文件



```c
#include "intrins.h"
sbit scl = P2 ^ 0;
sbit sda = P2 ^ 1;
```

自定义的读写函数

在IIC通信中，所有外设都挂在总线上，通过地址的最后一位（0为写，1为读）来决定操作类型。我们需要记住EEPROM的设备地址是0xA0（1010 0000），它挂在总线的第二个位置（而旁边的ADDA是0x90，即1001 0000，挂在第三个位置）。



IIC通信过程回顾

IIC通信基本流程：

1. 选择芯片并等待回复
2. 进行写入或读取操作
3. 停止通信

对于EEPROM操作：

- **写入数据**：先发送0xA0选择芯片，告知它准备从特定地址开始写入数据，然后依次写入数据
- **读取数据**：先发送0xA0选择芯片并指定读取地址，然后发送0xA1开始读取操作，读取完成后发送ACK指令1（拉高）结束传输

下面是读写函数的实现，两个函数都接收三个参数：

- `str`：存储/读取的字符串指针
- `addr`：存储/读取的起始地址（只需不超过总页数）
- `num`：需要存储/读取的数据个数

```
void EEPROM_Write(unsigned char *str, unsigned char addr, unsigned char num)
{
  I2CStart();
  I2CSendByte(0xa0);
  I2CWaitAck();
  I2CSendByte(addr);
  I2CWaitAck();

  while (num--)
  {
    I2CSendByte(*str++);
    I2CWaitAck();
    I2C_Delay(200);
  }
  I2CStop();
    //部分板子10个
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
}
void EEPROM_Read(unsigned char *str, unsigned char addr, unsigned char num)
{
  I2CStart();
  I2CSendByte(0xa0);
  I2CWaitAck();
  I2CSendByte(addr);
  I2CWaitAck();

  I2CStart();
```

```
  I2CSendByte(0xa1);
  I2CWaitAck();
  while (num--)
  {
    *str++ = I2CReceiveByte();
    if (num)
      I2CSendAck(0);
    else
      I2CSendAck(1);
  }
  I2CStop();
}
```

注意在Write函数末尾添加了6个delay255，这是为了保证写入时序正常，特别是在连续写入不同地址时避免写入异常。

完整代码

```
/*  #   I2C代码片段说明
        1.  本文件夹中提供的驱动代码供参赛选手完成程序设计参考。
        2.
   参赛选手可以自行编写相关代码或以该代码为基础，根据所选单片机类型、运行速度和试题
                中对单片机时钟频率的要求，进行代码调试和修改。
*/
#include "iic.h"

#include "intrins.h"
#define DELAY_TIME 10
sbit scl = P2 ^ 0;
sbit sda = P2 ^ 1;
//
//
static void I2C_Delay(unsigned char n)
{
    do
    {
        _nop_();_nop_();_nop_();_nop_();_nop_();
        _nop_();_nop_();_nop_();_nop_();_nop_();
        _nop_();_nop_();_nop_();_nop_();_nop_();
    }
    while(n--);
}

//
void I2CStart(void)
```

```
{
  sda = 1;
  scl = 1;
  I2C_Delay(DELAY_TIME);
  sda = 0;
  I2C_Delay(DELAY_TIME);
  scl = 0;
}

//
void I2CStop(void)
{
  sda = 0;
  scl = 1;
  I2C_Delay(DELAY_TIME);
  sda = 1;
  I2C_Delay(DELAY_TIME);
}

//
void I2CSendByte(unsigned char byt)
{
  unsigned char i;

  for (i = 0; i < 8; i++)
  {
    scl = 0;
    I2C_Delay(DELAY_TIME);
    if (byt & 0x80)
    {
      sda = 1;
    }
    else
    {
      sda = 0;
    }
    I2C_Delay(DELAY_TIME);
    scl = 1;
    byt <<= 1;
    I2C_Delay(DELAY_TIME);
  }

  scl = 0;
}

//
unsigned char I2CReceiveByte(void)
{
  unsigned char da;
  unsigned char i;
  for (i = 0; i < 8; i++)
```

```
  {
    scl = 1;
    I2C_Delay(DELAY_TIME);
    da <<= 1;
    if (sda)
      da |= 0x01;
    scl = 0;
    I2C_Delay(DELAY_TIME);
  }
  return da;
}

//
unsigned char I2CWaitAck(void)
{
  unsigned char ackbit;

  scl = 1;
  I2C_Delay(DELAY_TIME);
  ackbit = sda;
  scl = 0;
  I2C_Delay(DELAY_TIME);

  return ackbit;
}

//
void I2CSendAck(unsigned char ackbit)
{
  scl = 0;
  sda = ackbit;
  I2C_Delay(DELAY_TIME);
  scl = 1;
  I2C_Delay(DELAY_TIME);
  scl = 0;
  sda = 1;
  I2C_Delay(DELAY_TIME);
}

void EEPROM_Write(unsigned char *str, unsigned char addr,
unsigned char num)
{
  I2CStart();
  I2CSendByte(0xa0);
  I2CWaitAck();
  I2CSendByte(addr);
  I2CWaitAck();

  while (num--)
  {
    I2CSendByte(*str++);
```

```
    I2CWaitAck();
    I2C_Delay(200);
  }
  I2CStop();
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
}
void EEPROM_Read(unsigned char *str, unsigned char addr,
unsigned char num)
{
  I2CStart();
  I2CSendByte(0xa0);
  I2CWaitAck();
  I2CSendByte(addr);
  I2CWaitAck();

  I2CStart();
  I2CSendByte(0xa1);
  I2CWaitAck();
  while (num--)
  {
    *str++ = I2CReceiveByte();
    if (num)
      I2CSendAck(0);
    else
      I2CSendAck(1);
  }
  I2CStop();
}
```

EEPROM使用示例

EEPROM的使用主要有以下几种情况：

- 单字节存储（unsigned char）
- 多字节存储（unsigned int）
- 数组存储（unsigned char*）

值得注意的是，由于EEPROM是连续写入的，当我们存放/读取unsigned int数据时，它会自动占用两个连续位置。

```c
unsigned char arr_input[5]={1,2,3,4,5};
unsigned char arr_output[5];
EEPROM_Write(arr_input,0,5);
EEPROM_Read(arr_output,0,5);

unsigned char input = 5;
unsigned char output;
EEPROM_Write(&input,0,1);
EEPROM_Read(&output,0,1);

unsigned int input = 500;
unsigned int output;
EEPROM_Write(&input,0,2);
EEPROM_Read(&output,0,2);

float input_f=1.05555;
unsigned int input=input_f*100;//假设题目保留两位小数 input就是105
float output_f;
unsigned int output;
EEPROM_Write(&input,0,2);
EEPROM_Read(&output,0,2);
output_f=(float)output/100.0f;
```

EEPROM加锁机制

一、问题背景

1. **初始状态不确定性**
   - 新板载EEPROM初始值为随机值（非全255）
2. **数据污染风险**
   - 已写入数据的EEPROM无法自动清除
   - 未加锁机制可能导致未知数据干扰系统初始化

二、加锁机制设计

1. **核心原理**
   - 使用未用地址存储校验标记（Lock Byte）
   - 通过校验值判断EEPROM是否已初始化
2. **实现策略**

```
unsigned char EEPROM_Lock = 5;  // 校验标记值

void Key_Proc()
{
    Key_Val = Key_Read();
    Key_Down = Key_Val & (Key_Val ^ Key_Old);
    Key_Up = ~Key_Val & (Key_Val ^ Key_Old);
    Key_Old = Key_Val;
    switch (Key_Down)
    {
        case 12:
            AD_Para_10x = AD_Para_Ctrl_10x;
            EEPROM_Write(&AD_Para_10x, 0, 1);    // 写入参数
            EEPROM_Write(&EEPROM_Lock, 8, 1);    // 写入校验标记
            break;
    }
}
```

三、启动校验流程

```c
void main()
{
    unsigned char EEPROM_Temp;
    EEPROM_Read(&EEPROM_Temp, 8, 1);  // 读取校验标记
    // 校验通过则加载存储参数
    if (EEPROM_Temp == EEPROM_Lock)
    {
        EEPROM_Read(&AD_Para_10x, 0, 1);
    }
}
```

四、操作准则

- 必须启用加锁机制防止数据污染
- 校验地址应选择未使用的EEPROM空间

EEPROM连续读写注意事项

I2C时序要求

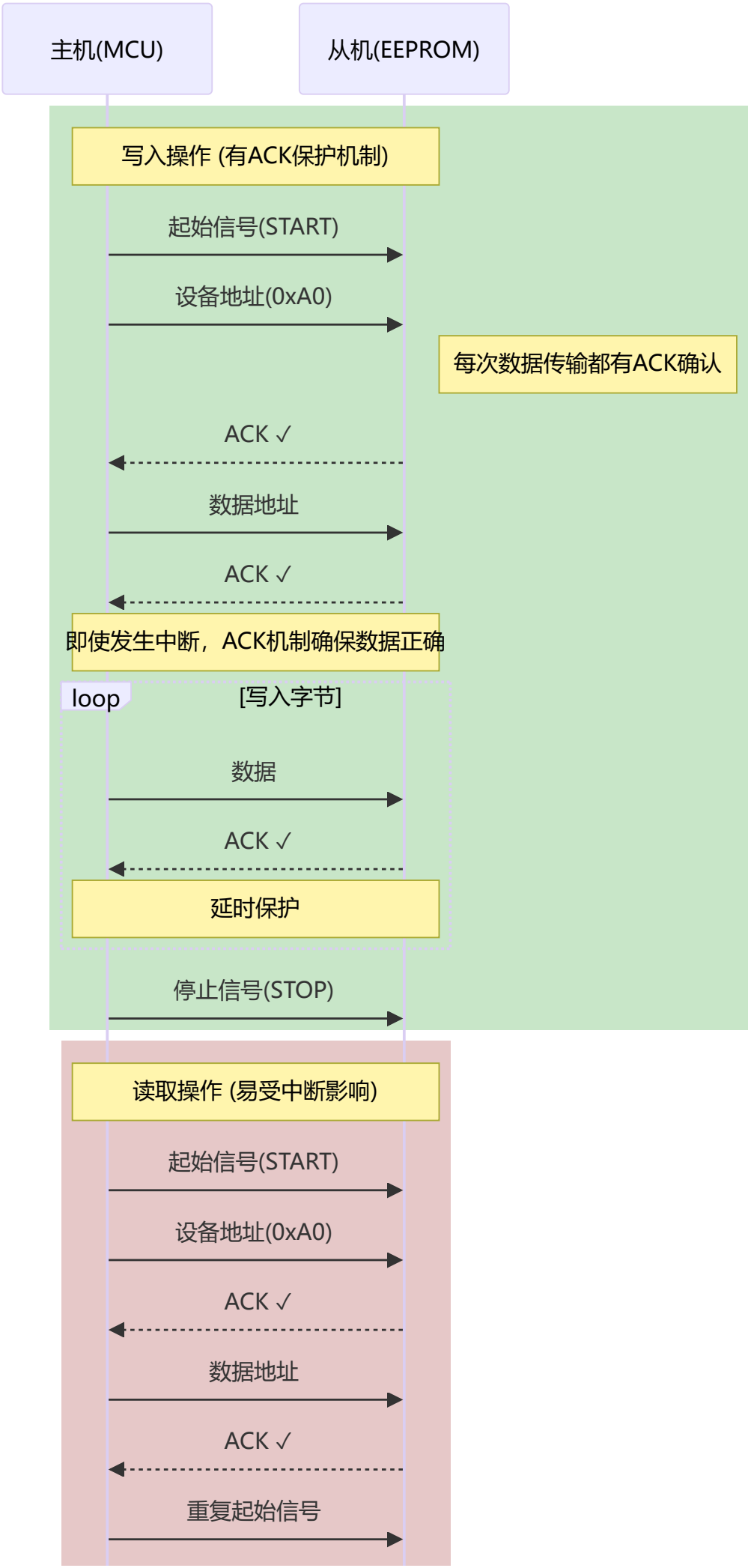
I2C通信对时序要求严格，在进行EEPROM读写操作时需要特别注意以下几点：

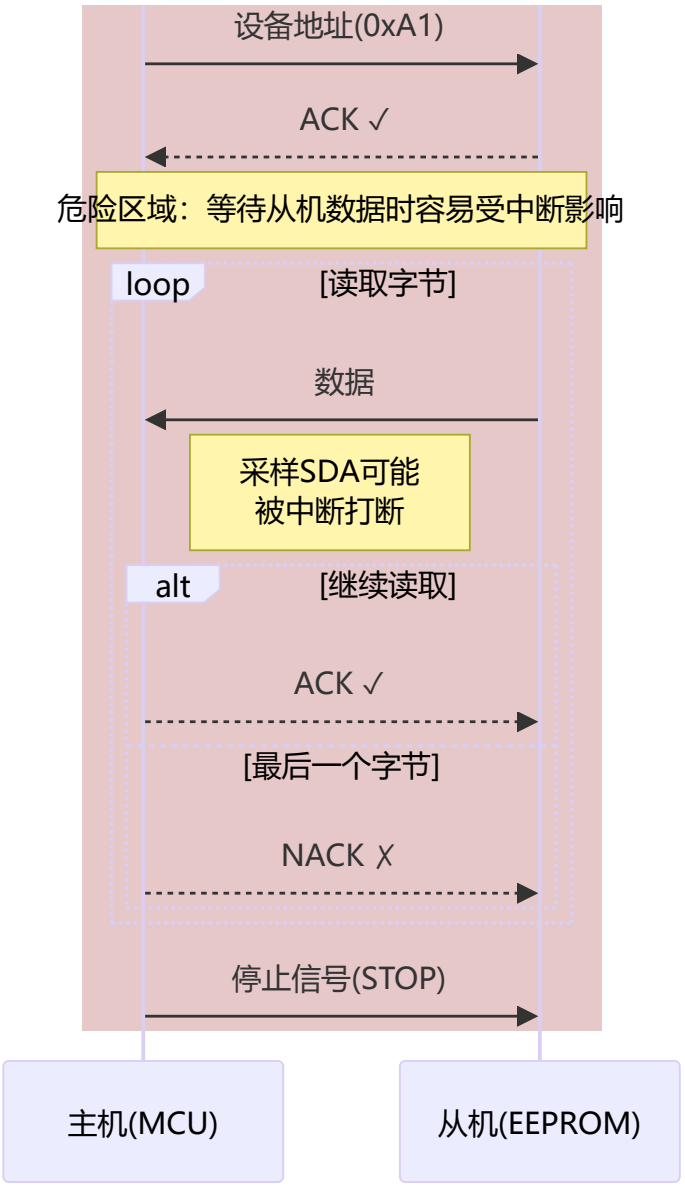
1. **初始化顺序**：必须在EEPROM上电读取操作完成后，再初始化定时器。否则定时器中断可能会破坏I2C的时序，导致读取失败。
2. **写入操作的可靠性**：
   - 写入过程由单片机(主机)主动控制
   - 每个字节都有ACK应答确认机制
   - 包含足够的延时保护
   - 即使发生中断，也不会影响数据写入的正确性

3. **读取操作的风险**：

   - 读取时需要等待EEPROM(从机)发送数据
   - I2C通过采样SDA线上的电平来判断数据
   - 如果在采样过程中发生中断，可能导致数据读取错误

I2C通信时序图


主机(MCU)
从机(EEPROM)
写入操作 (有ACK保护机制)
起始信号(START)
设备地址(0xA0)
每次数据传输都有ACK确认
ACK ✓
数据地址
ACK ✓
即使发生中断，ACK机制确保数据正确
loop
[写入字节]
数据
ACK ✓
延时保护
停止信号(STOP)
读取操作 (易受中断影响)
起始信号(START)
设备地址(0xA0)
ACK ✓
数据地址
ACK ✓
重复起始信号



代码实现

```c
void EEPROM_Write(unsigned char *EEPROM_String, unsigned
char addr, unsigned char num)
{
    // 选择芯片为EEPROM，模式为写
    I2CStart();
    I2CSendByte(0xA0);
    I2CWaitAck();

    I2CSendByte(addr); // 写入的数据的地址
    I2CWaitAck();
    while (num--)
    {
        I2CSendByte(*EEPROM_String++);
        I2CWaitAck();
```

```
        I2C_Delay(200);  // 单字节写入延时保护
    }
    I2CStop();
    // 写入完成后的延时保护
    I2C_Delay(255);
    I2C_Delay(255);
    I2C_Delay(255);
    I2C_Delay(255);
    I2C_Delay(255);
    I2C_Delay(255);
}
```

```
void EEPROM_Read(unsigned char *EEPROM_String, unsigned char addr, unsigned char num)
{
    // 第一阶段：设置读取地址
    I2CStart();
    I2CSendByte(0xA0);  // EEPROM写模式
    I2CWaitAck();
    I2CSendByte(addr);  // 设置起始地址
    I2CWaitAck();

    // 第二阶段：读取数据
    I2CStart();
    I2CSendByte(0xA1);  // EEPROM读模式
    I2CWaitAck();
    while (num--)
    {
        *EEPROM_String++ = I2CReceiveByte();
        I2CSendAck(num ? 0 : 1);  // 最后一个字节发送NACK
    }
    I2CStop();
}
```